SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	**CSA1209 & CSA1215**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO3**	**Assessment:**	**ASSESSMENT TOOL 1- EXPERIMENTS**
Weightage:	**40%**	**Total Marks:**	**25**
CO3 Statement:	*Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)*		
Student Name:	KAMALESH K	**Reg.No.:**	192572164
Department:	BE.CSE	**Date:**	

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct:

I KAMALESH K (192572164)*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1.


Online C Compiler
Share
Run
Output
#include <stdio.h>
typedef struct {
int R0;
int R1;
int R2;
int R3;
int R4;
} Registers;
void displayRegisters(Registers r) {
printf("\n--------------------------------\n");
printf("Register Status\n");
printf("--------------------------------\n");
printf("R0 = %d\n", r.R0);
printf("R1 = %d\n", r.R1);
printf("R2 = %d\n", r.R2);
printf("R3 = %d\n", r.R3);
printf("R4 = %d\n", r.R4);
}
int main() {
Registers r = {0, 0, 0, 0, 0};
int choice;
printf("=======================================\n");
printf(" REGISTER TRANSFER AND ALU SIMULATOR\n");
printf("=======================================\n");
=======================================
REGISTER TRANSFER AND ALU SIMULATOR
=======================================
Enter value for R1: Enter value for R2:

Register Status

R0 = 0
R1 = 20
R2 = 10
R3 = 0
R4 = 0
Input
20 10

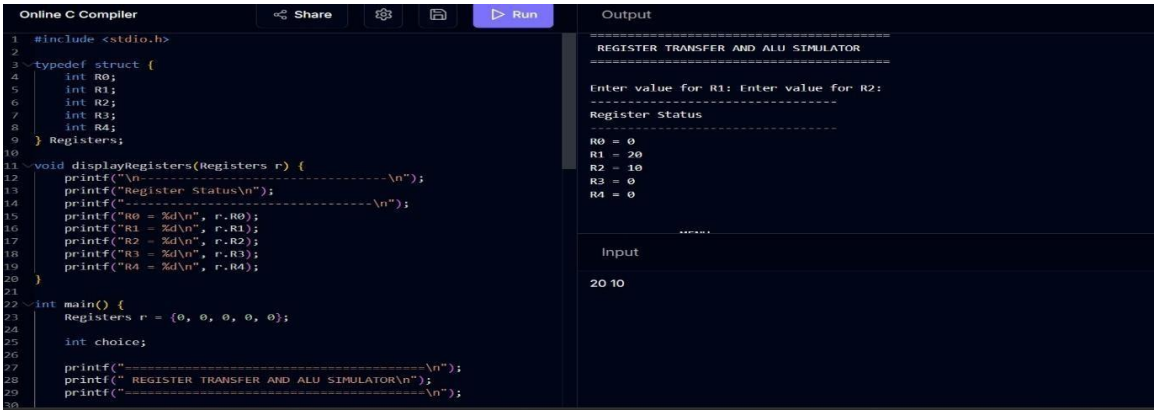
2.


Online C Compiler
Share
Run
Output
#include <stdio.h>
typedef struct {
int R0;
int R1;
int R2;
int R3;
int R4;
} Registers;
void displayRegisters(Registers r) {
printf("\n--------------------------------\n");
printf("Register Status\n");
printf("--------------------------------\n");
printf("R0 = %d\n", r.R0);
printf("R1 = %d\n", r.R1);
printf("R2 = %d\n", r.R2);
printf("R3 = %d\n", r.R3);
printf("R4 = %d\n", r.R4);
}
int main() {
Registers r = {0, 0, 0, 0, 0};
int choice;
printf("=======================================\n");
printf(" REGISTER TRANSFER AND ALU SIMULATOR\n");
printf("=======================================\n");
=======================================
REGISTER TRANSFER AND ALU SIMULATOR
=======================================
Enter value for R1: Enter value for R2:

Register Status

R0 = 0
R1 = 20
R2 = 10
R3 = 0
R4 = 0
Input
20 10

3.


Online C Compiler
Share
Run
Output
#include <stdio.h>
typedef struct {
int R0;
int R1;
int R2;
int R3;
int R4;
} Registers;
void displayRegisters(Registers r) {
printf("\n--------------------------------\n");
printf("Register Status\n");
printf("--------------------------------\n");
printf("R0 = %d\n", r.R0);
printf("R1 = %d\n", r.R1);
printf("R2 = %d\n", r.R2);
printf("R3 = %d\n", r.R3);
printf("R4 = %d\n", r.R4);
}
int main() {
Registers r = {0, 0, 0, 0, 0};
int choice;
printf("=======================================\n");
printf(" REGISTER TRANSFER AND ALU SIMULATOR\n");
printf("=======================================\n");
REGISTER TRANSFER AND ALU SIMULATOR
Enter value for R1: Enter value for R2:
Register Status
R0 = 0
R1 = 20
R2 = 10
R3 = 0
R4 = 0
Input
20 10

1.

4.

```c
#include <stdio.h>

typedef struct {
    int R1;
    int R2;
    int R3;
    int R4;
} Registers;

void singleBusSimulation() {

    Registers r = {10, 20, 0, 0};

    int cycles = 0;

    printf("\n=====================================\n");
    printf(" SINGLE-BUS ORGANIZATION\n");
    printf("=====================================\n");

    printf("\nInitial values:");
    printf("\nR1 = %d", r.R1);
    printf("\nR2 = %d", r.R2);

    printf("\n\nTransfer R1 -> R3");
    r.R3 = r.R1;
    cycles++;

    printf("\nTransfer R2 -> R4");
    r.R4 = r.R2;
    cycles++;
```

```
 SINGLE-BUS ORGANIZATION
=====================================

Initial values:
R1 = 10
R2 = 20

Transfer R1 -> R3
Transfer R2 -> R4
Memory -> R1
R2 -> I/O

Final Register Values:
R1 = 100
R2 = 20
```

Input

Enter your input here...

5.

Online C Compiler | Share | Run

```c
#include <stdio.h>

typedef struct {
    int R0;
    int R1;
    int R2;
    int R3;
    int R4;
} Registers;

void displayRegisters(Registers r) {
    printf("\n--------------------------------\n");
    printf("Register Status\n");
    printf("--------------------------------\n");
    printf("R0 = %d\n", r.R0);
    printf("R1 = %d\n", r.R1);
    printf("R2 = %d\n", r.R2);
    printf("R3 = %d\n", r.R3);
    printf("R4 = %d\n", r.R4);
}

int main() {
    Registers r = {0, 0, 0, 0, 0};

    int choice;

    printf("=====================================\n");
    printf(" REGISTER TRANSFER AND ALU SIMULATOR\n");
    printf("=====================================\n");
```

Output

```
=====================================
 REGISTER TRANSFER AND ALU SIMULATOR
=====================================

Enter value for R1: Enter value for R2:
--------------------------------
Register Status
--------------------------------
R0 = 0
R1 = 20
R2 = 10
R3 = 0
R4 = 0
```

Input

20 10